

CXL Emulation on QEMU via WSL2 Report

1. Introduction

CXL (Compute Express Link) is an open standard for high-speed, cache-coherent CPU-to-device and CPU-to-memory interconnects, built on top of the PCIe physical layer. Software emulation using QEMU remains the standard approach for CXL development and testing in the absence of accessible real hardware.

The purpose of this report is narrow and concrete: to demonstrate that CXL emulation via QEMU works successfully on a Windows machine, running Ubuntu 24.04 LTS under WSL2. It covers both acceleration paths available in this environment, hardware-accelerated KVM and software-emulated TCG, and the specific changes that made each of them work.

2. KVM vs. TCG on WSL2

QEMU can run its guest CPU either through KVM (hardware-assisted virtualization, using the host CPU's virtualization extensions directly) or through TCG (Tiny Code Generator, QEMU's software CPU emulator, used when hardware acceleration is unavailable). The two behave very differently for CXL emulation on WSL2:

Aspect	KVM	TCG
Availability on WSL2	Requires a WSL2 kernel built with CONFIG_KVM enabled, not present in the stock Microsoft WSL2 kernel	Available by default with any QEMU build; no kernel prerequisite
Speed	Near-native, guest instructions execute directly on host CPU where possible	Substantially slower, every guest instruction is translated and interpreted in software
CXL region creation (original report)	Not tested in the original report	Failed, HDM decoder commit error
CXL region creation (this update)	Succeeds reliably	Succeeds, once the guest-kernel mailbox timeout is increased (Section 3)
Setup requirement	WSL2 kernel rebuild with CONFIG_KVM (Section 4)	No special host/kernel prerequisite; only the guest-kernel timeout change is needed

The practical implication is that both paths are now usable on WSL2: KVM gives full speed once the WSL2 kernel is rebuilt, and TCG remains a viable fallback, including on machines where KVM cannot be enabled (e.g., virtualization not exposed to the WSL2 VM by the host, or nested-virtualization restrictions), provided the guest kernel's mailbox-ready timeout is increased as described below.

3. Guest-Kernel Timeout Fix, Making TCG Viable

The original report's TCG-based attempt failed at HDM decoder commit. Investigation traced this to a timeout in the Linux guest kernel's CXL driver: the kernel waits for the CXL device mailbox to signal ready (and for certain mailbox commands, such as those involved in decoder/region commit, to complete) within a fixed timeout window. Under KVM this window is comfortably long enough because guest instructions execute near-natively. Under TCG, every instruction is software-interpreted, so the same operation takes substantially longer in wall-clock time, long enough that the original, shorter timeout

value expired before the operation completed, and the kernel gave up and reported a failure, even though the operation would have succeeded given more time.

The fix applied was to increase this guest-kernel CXL mailbox-ready timeout. The CXL driver already exposes the relevant values as module parameters, which the earlier technical work in this project had already identified:

```
cxl_core    module parameter: media_ready_timeout
cxl_pci     module parameter: mbox_ready_timeout
```

Raising these timeout values in the guest kernel gives TCG's much slower instruction execution enough wall-clock time to complete the same mailbox handshake and decoder-commit sequence that KVM completes quickly by default. With the increased timeout in place, region creation and the rest of the CXL provisioning workflow (documented in the companion PMEM and VMEM reports) succeed under TCG as well as under KVM, the only practical difference being how long each step takes to finish.

This is the key correction to the original report's conclusion: TCG was never fundamentally incapable of CXL emulation on this stack, it was failing because a kernel-side timeout tuned for near-native (KVM) speeds was too short for software-emulated (TCG) speeds. Increasing that timeout resolves it.

4. Enabling KVM on WSL2 (When Available)

Where TCG needs only the timeout fix above, using KVM on WSL2 additionally requires a rebuilt WSL2 kernel, since the stock Microsoft WSL2 kernel does not ship with CONFIG_KVM enabled. This is handled by CXL_Complete_Setup.sh's WSL-only phase:

- Detects WSL vs. native Linux by checking /proc/version for 'microsoft'.
- Clones Microsoft's WSL2-Linux-Kernel repository (branch linux-msft-wsl-6.6.y).
- Starts from Microsoft's own config-wsl base config, then enables CONFIG_KVM, CONFIG_KVM_INTEL, and CONFIG_KVM_AMD.
- Builds the kernel, copies the resulting bzImage to the Windows filesystem, and writes a .wslconfig file pointing WSL2 at the new kernel via PowerShell automation.
- Shuts WSL2 down (wsl --shutdown) so the new kernel takes effect; the script is re-run afterward and detects the new kernel is already active, skipping straight past this phase.

This phase is entirely skipped on native Ubuntu, where the distribution kernel already has KVM enabled, it is a WSL2-only prerequisite, and only needed for the KVM path, not the TCG path.

5. Verifying KVM Is Actually Working

Before relying on accel=kvm in a QEMU launch command, CXL_Complete_Setup.sh checks (and the same commands can be run manually to confirm) that KVM is genuinely usable inside the WSL2 instance:

```
# Confirm the running kernel has KVM support loaded
lsmod | grep -q kvm

# Confirm the current user is in the kvm group (required for /dev/kvm access)
groups | grep kvm
```

```
# If KVM modules are not loaded, load them explicitly
sudo modprobe kvm
sudo modprobe kvm_intel 2>/dev/null || sudo modprobe kvm_amd 2>/dev/null || true
```

The setup script treats a positive `lsmod | grep -q kvm` result (or the presence of a previously written `.wslconfig` pointing at a KVM-enabled kernel) as evidence that Phase 2 (the WSL2 kernel rebuild) has already succeeded, and skips rebuilding the kernel again on subsequent runs. In practice, if either check fails, QEMU should be launched with `accel=tcg` instead of `accel=kvm`, and the guest-kernel timeout fix from Section 3 becomes relevant.

6. Environment and Setup

Item	Value
Host	Windows 11, WSL2 (Ubuntu 24.04 LTS)
Guest OS	Ubuntu 24.04.4 LTS (cloud image, cloud-init provisioned)
Guest Kernel	Linux v6.18, CXL subsystem enabled, with the mailbox-ready timeout increased for TCG viability
QEMU	Custom build from the jic23 CXL fork (branch <code>cxl-2025-03-20</code>), built by <code>CXL_Complete_Setup.sh</code>
Acceleration	KVM (after WSL2 kernel rebuild) or TCG (with the timeout fix), both verified working
CXL Tooling	<code>cxl-cli</code> (<code>cxl</code>), <code>ndctl</code> , <code>pciutils</code>

QEMU itself was built with a small number of CXL-related source patches applied by `CXL_Complete_Setup.sh` (covering RAS error-mask handling and a mailbox command addition); these are not the subject of this report and are documented separately alongside the PMEM/VMEM error-injection discussion.

7. Dual-Switch Tracks Validated on WSL2

With both acceleration paths confirmed working, the two dual-switch topologies documented in the companion PMEM and VMEM reports were run and validated on this WSL2 environment:

7.1 Persistent Memory Track

Two independent pools, four 1 GiB Type 3 pmem devices across two CXL switches. Regions, namespaces, ext4 filesystems, and DAX mounts were provisioned successfully, with checksum-verified read/write I/O confirming both data integrity and independence between the two pools.

7.2 Volatile Memory Track

Four 1 GiB Type 3 volatile devices across two CXL switches, onlined into general System RAM against a deliberately constrained 1 GiB baseline. A 1.5 GiB stress-ng allocation against that baseline completed without an OOM or crash, confirming the CXL-backed capacity is genuinely usable as System RAM.

7.3 Verified Results, Original Report vs. This Update

Check	Original WSL2 Report (TCG, untuned timeout)	This Update (KVM and TCG, both working)
CXL device discovery	Yes, single 512 MiB device	Yes, up to 8 devices across two switches per track, on either acceleration path
HDM decoder commit / region creation	No, failed under TCG	Yes, succeeds under KVM directly, and under TCG once the mailbox-ready timeout is increased
Namespace / DAX usage	No, blocked by region failure	Yes, fsdax namespaces, ext4, DAX mount with fallback (pmem track)
CXL memory as usable RAM	No	Yes, verified via 1.5 GiB spillover stress test (vmem track)
KVM availability on WSL2	Not addressed	Verified via lsmod/groups checks; enabled via a WSL2 kernel rebuild when absent
TCG viability	Assumed broken	Confirmed working once the guest-kernel timeout is increased

8. Known Limitations

- TCG remains substantially slower than KVM for the same workflow; the timeout fix makes TCG succeed, not fast, it should be treated as a fallback path, not a preferred one, when KVM is available.
- Global Persistent Flush / dirty-shutdown state remains unemulated by QEMU's cxl_nvdimm backend on the pmem track, benign in emulation, unrelated to the KVM/TCG distinction.
- Interleaved volatile regions still trigger a KVM MMIO limitation (Suberror 3) on the volatile track; a 1-way-region workaround is used regardless of acceleration backend.
- RAS-specific QEMU behavior (error-mask patches, CCI/QMP error injection) is out of scope for this report and is covered in the companion PMEM document.